Lab Handout 1-H
Field-Programmable Gate Array (FPGA) Universal Asynchronous Receiver/Transmitter (UART) Ingestion and End-to-End Data Flow

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Implement a minimal UART ingestion path on the Nexys A7-100T that receives the 43-byte forwarding frames from the STM32 interceptor (LH1-G), detects the start delimiter, extracts the Beacon ID, and displays it on the seven-segment display. This closes the Module 1 end-to-end data path: **cooperative target → RF link → interceptor (Threat) → FPGA**.

2 Prerequisites

- Lab Handouts 1-E (Vivado toolchain), 1-F (cooperative target firmware), and 1-G (interceptor / Threat firmware) complete.
- The interceptor node from LH1-G must be producing valid frames on `USART3 TX` (`PC4`). If you skipped LH1-G Step 7 because no USB-UART adapter was available, this handout is the validation step — the FPGA seven-segment display will count frames in lieu of the host-side parser.
- LH1-E loopback projects *do not* need to be open; this handout creates a separate Vivado project (`ingest_top`) that lives alongside them.

3 Architecture

Block	Function
`uart_rx`	115200 8N1 UART receiver (16× oversampled)
`byte_fifo`	16-deep × 8-bit synchronous First In, First Out (FIFO) between `uart_rx` and frame detector
`frame_detector`	Synchronize on `0x7E` start, capture 43 bytes, expose Beacon ID
`seg7_driver`	Drive the four right-most digits with the Beacon ID (hex) and a 16-bit frame counter
`ingest_top`	Top-level wiring; instantiates all four blocks

[Figure placeholder]

`LH1H_block_diagram.png`

FPGA ingestion block diagram: `uart_rx` → `byte_fifo` → `frame_detector` → `seg7_driver`. Key signals (`rx_strobe`, `beacon_id`, `new_frame`) are labeled on the data path.

Figure 1: FPGA ingestion block diagram: `uart_rx` → `byte_fifo` → `frame_detector` → `seg7_driver`. Key signals (`rx_strobe`, `beacon_id`, `new_frame`) are labeled on the data path.

4 Wiring

Connect the Threat's `USART3 TX` pin to one of the Nexys A7 PMOD input pins. Common ground is essential.

From (Threat Nucleo)	Function	To (Nexys A7)	FPGA Pin
Morpho `PC4` (USART3_TX)	3.3 V serial out, 115200 8N1	PMOD JA pin 1 (`JA[0]`)	`C17`
Morpho `GND`	Common ground	PMOD JA pin 5 (or 11)	—

[Figure placeholder]

`LH1H_pmod_wiring.png`

STM32 `USART3 TX` (`PC4`) wired to Nexys A7 PMOD JA pin 1 (FPGA pin `C17`, called `JA[0]` in the master XDC) with common ground on PMOD JA pin 5. Annotated photo clarifies which PMOD connector is in use.

Figure 2: STM32 `USART3 TX` (`PC4`) wired to Nexys A7 PMOD JA pin 1 (FPGA pin `C17`, called `JA[0]` in the master XDC) with common ground on PMOD JA pin 5. Annotated photo clarifies which PMOD connector is in use.

Voltage Levels and Connector Choice

The STM32 drives 3.3 V logic and the Nexys A7 PMOD pins are 3.3 V native. No level shifter is required. *Use PMOD JA, not JC* — some Nexys A7 board revisions wire JC (the high-speed PMOD next to the FMC connector) to a different I/O bank, and the LVDS-style pin pairs there have additional series resistors that can corrupt single-ended UART. JA, JB, and JD are all safe.

5 Procedure

5.1 Step 1 — Create the Vivado project

Step 1.1 — Launch Vivado and start a new project

1. Launch Vivado from the Start Menu (Windows) or the desktop launcher (Linux). The default Vivado start page opens.
2. Click **Quick Start** → **Create Project**. The New Project wizard opens.
3. Click **Next** on the wizard's first page.
4. On the **Project Name** page:
   - Project name: `ingest_top`
   - Project location: a local-only folder *not* inside OneDrive, iCloud Drive, Dropbox, or Google Drive (Vivado writes hundreds of intermediate files; cloud sync corrupts them mid-write).
   - Tick **Create project subdirectory**.

   Click **Next**.
5. On the **Project Type** page, select **RTL Project**. Tick **Do not specify sources at this time** (you will add them in Step 2). Click **Next**.
6. On the **Default Part** page, click the **Boards** tab (not Parts), type `Nexys A7-100T` in the search box, select the Digilent Nexys A7-100T row, and click **Next**.
   - If the board is not listed, your Vivado install is missing the Digilent board files. Stop, install them per the LH1-E Step 1 *Common Pitfalls* entry, and restart this handout.
   - Confirm the right-hand **Selected Part** field shows `xc7a100tcsg324-1`.
7. On the **New Project Summary** page, click **Finish**.

Step 1.2 — Verify the project opened correctly

1. The Vivado IDE should open with the Project Manager view active. The **Sources** panel (left side) is empty under *Design Sources*, *Constraints*, and *Simulation Sources*.
2. Confirm the title bar reads `ingest_top - [<your-path>/ingest_top.xpr] - Vivado <version>`.
3. Confirm the bottom-right of the IDE shows part `xc7a100tcsg324-1`.

5.2 Step 2 — Add the four VHSIC Hardware Description Language (VHDL) source files

You will create five VHDL files: four block files (`uart_rx.vhd`, `byte_fifo.vhd`, `frame_detector.vhd`, `seg7_driver.vhd`) and one top-level (`ingest_top.vhd`). Create them one at a time using the same procedure.

Step 2.1 — Create `uart_rx.vhd`

1. In the Sources panel, click the + button (Add Sources). Select **Add or create design sources** and click **Next**.
2. Click **Create File**. In the popup:
   - File type: **VHDL**
   - File name: `uart_rx`
   - File location: <**Local to Project**>

   Click **OK** then **Finish**. Dismiss the "Define Module" popup with **OK** (you will write the entity manually).
3. Double-click `uart_rx.vhd` in the Sources panel to open it in the editor. Replace its entire contents with the listing below. Save with `Ctrl+S`.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity uart_rx is
    generic (
        CLK_FREQ_HZ : integer := 100_000_000;
        BAUD        : integer := 115_200
    );
    port (
        clk         : in  std_logic;
        rst_n       : in  std_logic;
        rx          : in  std_logic;
        rx_data     : out std_logic_vector(7 downto 0);
        rx_strobe   : out std_logic
    );
end uart_rx;

architecture rtl of uart_rx is
    constant DIV16 : integer := CLK_FREQ_HZ / (BAUD * 16);  -- ~54 at 100 MHz
    signal os_cnt : unsigned(15 downto 0) := (others => '0');
    signal os_tick : std_logic := '0';

    type state_t is (S_IDLE, S_START, S_DATA, S_STOP);
    signal state    : state_t := S_IDLE;
    signal bit_idx  : unsigned(2 downto 0) := (others => '0');
    signal sub_cnt  : unsigned(3 downto 0) := (others => '0');
    signal shifter  : std_logic_vector(7 downto 0) := (others => '0');
```

```
    signal rx_sync1, rx_sync2 : std_logic := '1';
begin
    -- Two-flop synchroniser for the async rx pin
    process (clk) begin
        if rising_edge(clk) then
            rx_sync1 <= rx;
            rx_sync2 <= rx_sync1;
        end if;
    end process;

    -- 16x oversample tick generator
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                os_cnt  <= (others => '0');
                os_tick <= '0';
            elsif os_cnt = to_unsigned(DIV16 - 1, os_cnt'length) then
                os_cnt  <= (others => '0');
                os_tick <= '1';
            else
                os_cnt  <= os_cnt + 1;
                os_tick <= '0';
            end if;
        end if;
    end process;

    -- Receive FSM, advanced on os_tick
    process (clk) begin
        if rising_edge(clk) then
            rx_strobe <= '0';
            if rst_n = '0' then
                state   <= S_IDLE;
                sub_cnt <= (others => '0');
                bit_idx <= (others => '0');
            elsif os_tick = '1' then
                case state is
                    when S_IDLE =>
                        if rx_sync2 = '0' then           -- start bit
                            state   <= S_START;
                            sub_cnt <= to_unsigned(0, 4);
                        end if;
                    when S_START =>
                        if sub_cnt = to_unsigned(7, 4) then  -- mid-start
                            if rx_sync2 = '0' then       -- still low
                                state   <= S_DATA;
                                sub_cnt <= (others => '0');
                                bit_idx <= (others => '0');
                            else
                                state <= S_IDLE;          -- glitch
                            end if;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                    when S_DATA =>
```

```
                        if sub_cnt = to_unsigned(15, 4) then
                            shifter <= rx_sync2 & shifter(7 downto 1);
                            sub_cnt <= (others => '0');
                            if bit_idx = to_unsigned(7, 3) then
                                state <= S_STOP;
                            else
                                bit_idx <= bit_idx + 1;
                            end if;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                    when S_STOP =>
                        if sub_cnt = to_unsigned(15, 4) then
                            rx_data   <= shifter;
                            rx_strobe <= '1';
                            state     <= S_IDLE;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                end case;
            end if;
        end if;
    end process;
end rtl;
```

Listing 1: `uart_rx.vhd` — 115200 8N1 receiver, 16× oversampled. Generic `CLK_FREQ_HZ` and `BAUD` let you reuse the block at other rates.

Step 2.2 — Create `byte_fifo.vhd`

Repeat the Add Sources → Create File flow with **File name:** `byte_fifo`. Replace contents with:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity byte_fifo is
    generic (DEPTH_LOG2 : integer := 4);   -- 2**4 = 16 entries
    port (
        clk      : in  std_logic;
        rst_n    : in  std_logic;
        wr_en    : in  std_logic;
        wr_data  : in  std_logic_vector(7 downto 0);
        rd_en    : in  std_logic;
        rd_data  : out std_logic_vector(7 downto 0);
        empty    : out std_logic;
        full     : out std_logic
    );
end byte_fifo;

architecture rtl of byte_fifo is
    constant DEPTH : integer := 2**DEPTH_LOG2;
    type   mem_t is array (0 to DEPTH - 1) of std_logic_vector(7 downto 0)
        ;
```

```
    signal mem    : mem_t := (others => (others => '0'));
    signal w_ptr  : unsigned(DEPTH_LOG2 downto 0) := (others => '0');
    signal r_ptr  : unsigned(DEPTH_LOG2 downto 0) := (others => '0');
    signal w_idx  : unsigned(DEPTH_LOG2 - 1 downto 0);
    signal r_idx  : unsigned(DEPTH_LOG2 - 1 downto 0);
begin
    w_idx <= w_ptr(DEPTH_LOG2 - 1 downto 0);
    r_idx <= r_ptr(DEPTH_LOG2 - 1 downto 0);
    empty <= '1' when w_ptr = r_ptr else '0';
    full  <= '1' when (w_ptr(DEPTH_LOG2) /= r_ptr(DEPTH_LOG2)) and (w_idx
       = r_idx) else '0';

    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                w_ptr <= (others => '0');
                r_ptr <= (others => '0');
            else
                if wr_en = '1' and (w_ptr(DEPTH_LOG2) = r_ptr(DEPTH_LOG2)
                   or w_idx /= r_idx) then
                    mem(to_integer(w_idx)) <= wr_data;
                    w_ptr <= w_ptr + 1;
                end if;
                if rd_en = '1' and w_ptr /= r_ptr then
                    r_ptr <= r_ptr + 1;
                end if;
            end if;
            rd_data <= mem(to_integer(r_idx));
        end if;
    end process;
end rtl;
```

Listing 2: `byte_fifo.vhd` — 16-entry × 8-bit synchronous FIFO. Hand-rolled (no Xilinx primitive) so the file is portable to other parts.

Step 2.3 — Create `frame_detector.vhd`

Repeat the Add Sources flow with **File name:** `frame_detector`. Replace contents with:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity frame_detector is
    port (
        clk         : in  std_logic;
        rst_n       : in  std_logic;
        fifo_empty  : in  std_logic;
        fifo_data   : in  std_logic_vector(7 downto 0);
        fifo_rd_en  : out std_logic;
        beacon_id   : out std_logic_vector(7 downto 0);
        new_frame   : out std_logic
    );
end frame_detector;
```

```

architecture rtl of frame_detector is
    type state_t is (S_IDLE, S_LEN, S_PAYLOAD, S_DONE);
    signal state    : state_t := S_IDLE;
    signal byte_cnt : unsigned(5 downto 0) := (others => '0');  -- 0..40
    signal id_reg   : std_logic_vector(7 downto 0) := (others => '0');
begin
    process (clk) begin
        if rising_edge(clk) then
            fifo_rd_en <= '0';
            new_frame  <= '0';

            if rst_n = '0' then
                state    <= S_IDLE;
                byte_cnt <= (others => '0');
            elsif fifo_empty = '0' then
                fifo_rd_en <= '1';      -- pop one byte this cycle
                case state is
                    when S_IDLE =>
                        if fifo_data = x"7E" then
                            state <= S_LEN;
                        end if;
                    when S_LEN =>
                        if fifo_data = x"29" then        -- length = 41
                            state    <= S_PAYLOAD;
                            byte_cnt <= (others => '0');
                        else
                            state <= S_IDLE;             -- bad length
                        end if;
                    when S_PAYLOAD =>
                        -- The 41 payload bytes are frame[1..41].  We have already
                        -- consumed frame[1] (length).  We now consume the 40
                        -- remaining bytes 2..41.  The Beacon~ID lives at frame
                        -- offset 12 = radio-packet byte 2; the first PAYLOAD
                        -- byte we see here is frame offset 2 (timestamp byte 0),
                        -- so we capture when byte_cnt = 10 (12 - 2).
                        if byte_cnt = to_unsigned(10, 6) then
                            id_reg <= fifo_data;
                        end if;
                        if byte_cnt = to_unsigned(39, 6) then  -- last byte (frame[41])
                            state <= S_DONE;
                        else
                            byte_cnt <= byte_cnt + 1;
                        end if;
                    when S_DONE =>
                        beacon_id <= id_reg;
                        new_frame <= '1';
                        state     <= S_IDLE;
```

```
                            fifo_rd_en <= '0';    -- skip the trailing CRC-8 byte? no:
                                                  -- the CRC-8 byte (frame[42]) will be
                                                  -- silently consumed next cycle by
                                                  -- IDLE-state non-0x7E discard.
                    end case;
                end if;
            end if;
        end process;
end rtl;
```

Listing 3: `frame_detector.vhd` — IDLE/LEN/PAYLOAD/DONE FSM. Captures the radio-packet's Beacon ID (frame offset 12) into `beacon_id` and pulses `new_frame` once per valid frame.

Why `S_IDLE` silently discards

The frame CRC-8 byte at frame offset 42 is not validated in Module 1 — the IDLE state's "pop and check for 0x7E" behavior naturally discards any byte that is not a delimiter, so a valid CRC byte that happens not to be 0x7E falls through harmlessly. If the CRC byte happens to equal 0x7E (which occurs ∼0.4 % of frames), the FSM mis-aligns by one frame; on the next bad-length byte at S_LEN it resyncs. This cost is acceptable for the LH1-H end-to-end demo. Module 2 LH2-D adds a proper CRC check and an explicit consume-and-discard byte at the end of each frame.

Step 2.4 — Create `seg7_driver.vhd`

Repeat the Add Sources flow with **File name:** `seg7_driver`. Replace contents with:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity seg7_driver is
    generic (CLK_FREQ_HZ : integer := 100_000_000);
    port (
        clk        : in  std_logic;
        rst_n      : in  std_logic;
        beacon_id  : in  std_logic_vector(7 downto 0);
        new_frame  : in  std_logic;
        seg        : out std_logic_vector(6 downto 0);   -- a..g, active-low
        dp         : out std_logic;                       -- decimal point, active-low
        an         : out std_logic_vector(7 downto 0)     -- digit anodes, active-low
    );
end seg7_driver;

architecture rtl of seg7_driver is
    -- 4 kHz refresh => 1 kHz per digit
```

```
    constant REFRESH_DIV : integer := CLK_FREQ_HZ / 4_000;
    signal ref_cnt : unsigned(15 downto 0) := (others => '0');
    signal digit_sel : unsigned(1 downto 0) := (others => '0');
    signal frame_ctr : unsigned(15 downto 0) := (others => '0');
    signal nibble    : std_logic_vector(3 downto 0);

    function hex7(n : std_logic_vector(3 downto 0)) return
       std_logic_vector is
    begin
        case n is
            when x"0" => return "1000000";  when x"1" => return "1111001";
            when x"2" => return "0100100";  when x"3" => return "0110000";
            when x"4" => return "0011001";  when x"5" => return "0010010";
            when x"6" => return "0000010";  when x"7" => return "1111000";
            when x"8" => return "0000000";  when x"9" => return "0010000";
            when x"A" => return "0001000";  when x"B" => return "0000011";
            when x"C" => return "1000110";  when x"D" => return "0100001";
            when x"E" => return "0000110";  when x"F" => return "0001110";
            when others => return "1111111";
        end case;
    end function;
begin
    dp <= '1';                              -- decimal point off

    -- Frame counter: increments on every new_frame pulse
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                frame_ctr <= (others => '0');
            elsif new_frame = '1' then
                frame_ctr <= frame_ctr + 1;
            end if;
        end if;
    end process;

    -- Digit-select counter
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                ref_cnt <= (others => '0');
                digit_sel <= (others => '0');
            elsif ref_cnt = to_unsigned(REFRESH_DIV - 1, ref_cnt'length)
               then
                ref_cnt <= (others => '0');
                digit_sel <= digit_sel + 1;
            else
                ref_cnt <= ref_cnt + 1;
            end if;
        end if;
    end process;

    -- Choose nibble per digit and drive anodes
    process (digit_sel, beacon_id, frame_ctr) begin
        an <= (others => '1');            -- all off (active-low)
```

```
        case digit_sel is
            when "00" =>                    -- AN0: low nibble of beacon_id
                nibble <= beacon_id(3 downto 0);
                an(0) <= '0';
            when "01" =>                    -- AN1: high nibble of beacon_id
                nibble <= beacon_id(7 downto 4);
                an(1) <= '0';
            when "10" =>                    -- AN2: low nibble of frame_ctr
                nibble <= std_logic_vector(frame_ctr(3 downto 0));
                an(2) <= '0';
            when others =>                  -- AN3: next nibble of frame_ctr
                nibble <= std_logic_vector(frame_ctr(7 downto 4));
                an(3) <= '0';
        end case;
    end process;

    seg <= hex7(nibble);
end rtl;
```

Listing 4: `seg7_driver.vhd` — multiplexes the four right-most digits at ∼1 kHz per digit. Right two digits show the live Beacon ID; left two digits show a 16-bit frame counter modulo 256.

Step 2.5 — Create `ingest_top.vhd` and set as top

Repeat the Add Sources flow one last time with **File name:** `ingest_top`. Replace contents with:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ingest_top is
    port (
        clk100   : in  std_logic;
        cpu_resetn : in  std_logic;          -- active-low pushbutton
        uart_rx_pin : in  std_logic;         -- from Threat USART3 TX
        seg      : out std_logic_vector(6 downto 0);
        dp       : out std_logic;
        an       : out std_logic_vector(7 downto 0);
        led      : out std_logic_vector(15 downto 0)
    );
end ingest_top;

architecture rtl of ingest_top is
    signal rst_n      : std_logic;
    signal rx_data    : std_logic_vector(7 downto 0);
    signal rx_strobe  : std_logic;
    signal fifo_empty : std_logic;
    signal fifo_full  : std_logic;
    signal fifo_rd    : std_logic;
    signal fifo_q     : std_logic_vector(7 downto 0);
    signal beacon_id  : std_logic_vector(7 downto 0) := (others => '0');
    signal new_frame  : std_logic;
begin
    rst_n <= cpu_resetn;

```

```
    -- Heartbeat / status LEDs
    led(0)  <= rx_strobe;
    led(1)  <= new_frame;
    led(2)  <= fifo_empty;
    led(3)  <= fifo_full;
    led(15 downto 4) <= (others => '0');

    u_uart : entity work.uart_rx
        generic map (CLK_FREQ_HZ => 100_000_000, BAUD => 115_200)
        port map (
            clk => clk100, rst_n => rst_n, rx => uart_rx_pin,
            rx_data => rx_data, rx_strobe => rx_strobe
        );

    u_fifo : entity work.byte_fifo
        port map (
            clk => clk100, rst_n => rst_n,
            wr_en => rx_strobe, wr_data => rx_data,
            rd_en => fifo_rd, rd_data => fifo_q,
            empty => fifo_empty, full => fifo_full
        );

    u_det : entity work.frame_detector
        port map (
            clk => clk100, rst_n => rst_n,
            fifo_empty => fifo_empty, fifo_data => fifo_q,
            fifo_rd_en => fifo_rd,
            beacon_id => beacon_id, new_frame => new_frame
        );

    u_seg : entity work.seg7_driver
        generic map (CLK_FREQ_HZ => 100_000_000)
        port map (
            clk => clk100, rst_n => rst_n,
            beacon_id => beacon_id, new_frame => new_frame,
            seg => seg, dp => dp, an => an
        );
end rtl;
```

Listing 5: `ingest_top.vhd` — top-level wiring. After saving, right-click `ingest_top` in the Sources panel and choose **Set as Top**; Vivado will mark it bold.

1. In the Sources panel, right-click `ingest_top` and select **Set as Top**. The entity name should now appear in bold.
2. Confirm the Sources hierarchy expands to show `ingest_top` at top with `u_uart`, `u_fifo`, `u_det`, `u_seg` as children. If any of them shows a yellow "?" or red "!" marker, you have a typo in the corresponding file — fix before continuing.

5.3 Step 3 — Add and configure the constraint file

1. In the Sources panel, click + → **Add or create constraints** → **Add Files**. Navigate to your `Nexys-A7-100T-Master.xdc` from LH1-E Step 2, ensure **Copy sources into project** is ticked, click **Finish**.
2. Double-click the copied `Nexys-A7-100T-Master.xdc` under *Constraints* to open it.
3. Uncomment *exactly* the lines below by removing the leading `#` on each. All other lines in the master XDC must remain commented out.

```
## 100 MHz clock (oscillator on E3)
set_property -dict { PACKAGE_PIN E3   IOSTANDARD LVCMOS33 } [get_ports { clk100 }];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk100 }];

## CPU reset pushbutton (active-low; on the red board pushbutton near the JTAG header)
set_property -dict { PACKAGE_PIN C12  IOSTANDARD LVCMOS33 } [get_ports { cpu_resetn }];

## PMOD JA pin 1 = uart_rx_pin (from Threat USART3_TX = PC4)
set_property -dict { PACKAGE_PIN C17  IOSTANDARD LVCMOS33 } [get_ports { uart_rx_pin }];

## Seven-segment display (active-low segments and anodes)
set_property -dict { PACKAGE_PIN T10  IOSTANDARD LVCMOS33 } [get_ports { seg[0] }];
set_property -dict { PACKAGE_PIN R10  IOSTANDARD LVCMOS33 } [get_ports { seg[1] }];
set_property -dict { PACKAGE_PIN K16  IOSTANDARD LVCMOS33 } [get_ports { seg[2] }];
set_property -dict { PACKAGE_PIN K13  IOSTANDARD LVCMOS33 } [get_ports { seg[3] }];
set_property -dict { PACKAGE_PIN P15  IOSTANDARD LVCMOS33 } [get_ports { seg[4] }];
set_property -dict { PACKAGE_PIN T11  IOSTANDARD LVCMOS33 } [get_ports { seg[5] }];
set_property -dict { PACKAGE_PIN L18  IOSTANDARD LVCMOS33 } [get_ports { seg[6] }];
set_property -dict { PACKAGE_PIN H15  IOSTANDARD LVCMOS33 } [get_ports { dp }];

set_property -dict { PACKAGE_PIN J17  IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
set_property -dict { PACKAGE_PIN J18  IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
set_property -dict { PACKAGE_PIN T9   IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
set_property -dict { PACKAGE_PIN J14  IOSTANDARD LVCMOS33 } [get_ports { an[3] }];
set_property -dict { PACKAGE_PIN P14  IOSTANDARD LVCMOS33 } [get_ports { an[4] }];
```

```
set_property -dict { PACKAGE_PIN T14  IOSTANDARD LVCMOS33 } [get_ports {
    an[5] }];
set_property -dict { PACKAGE_PIN K2   IOSTANDARD LVCMOS33 } [get_ports {
    an[6] }];
set_property -dict { PACKAGE_PIN U13  IOSTANDARD LVCMOS33 } [get_ports {
    an[7] }];

## Status LEDs (only led[0..3] are driven; the rest are tied low)
set_property -dict { PACKAGE_PIN H17  IOSTANDARD LVCMOS33 } [get_ports {
    led[0] }];
set_property -dict { PACKAGE_PIN K15  IOSTANDARD LVCMOS33 } [get_ports {
    led[1] }];
set_property -dict { PACKAGE_PIN J13  IOSTANDARD LVCMOS33 } [get_ports {
    led[2] }];
set_property -dict { PACKAGE_PIN N14  IOSTANDARD LVCMOS33 } [get_ports {
    led[3] }];
set_property -dict { PACKAGE_PIN R18  IOSTANDARD LVCMOS33 } [get_ports {
    led[4] }];
set_property -dict { PACKAGE_PIN V17  IOSTANDARD LVCMOS33 } [get_ports {
    led[5] }];
set_property -dict { PACKAGE_PIN U17  IOSTANDARD LVCMOS33 } [get_ports {
    led[6] }];
set_property -dict { PACKAGE_PIN U16  IOSTANDARD LVCMOS33 } [get_ports {
    led[7] }];
set_property -dict { PACKAGE_PIN V16  IOSTANDARD LVCMOS33 } [get_ports {
    led[8] }];
set_property -dict { PACKAGE_PIN T15  IOSTANDARD LVCMOS33 } [get_ports {
    led[9] }];
set_property -dict { PACKAGE_PIN U14  IOSTANDARD LVCMOS33 } [get_ports {
    led[10] }];
set_property -dict { PACKAGE_PIN T16  IOSTANDARD LVCMOS33 } [get_ports {
    led[11] }];
set_property -dict { PACKAGE_PIN V15  IOSTANDARD LVCMOS33 } [get_ports {
    led[12] }];
set_property -dict { PACKAGE_PIN V14  IOSTANDARD LVCMOS33 } [get_ports {
    led[13] }];
set_property -dict { PACKAGE_PIN V12  IOSTANDARD LVCMOS33 } [get_ports {
    led[14] }];
set_property -dict { PACKAGE_PIN V11  IOSTANDARD LVCMOS33 } [get_ports {
    led[15] }];
```

Listing 6: Active constraint lines for `ingest_top`. Copy these exact `set_property` statements into your XDC.

Save the XDC (`Ctrl+S`).

Pin numbers are board-revision specific

The pin assignments above are correct for the Nexys A7-100T (Rev. B and later, the production silkscreen). Older A7-100T prototypes and the A7-50T variant differ on a handful of pins (notably the seg7 anodes). If *any* pin name above does not appear as a commented-out line in your master XDC, your master XDC is for a different board revision — re-download from Digilent's reference page using the exact SKU printed on your board.

5.4 Step 4 — Synthesize, implement, and program

1. Flow Navigator → **Run Synthesis**. Wait for completion. Expect zero errors. Warnings about "input port `rx` has no driver during simulation" on the standalone `uart_rx` entity are normal — it is driven by the top.
2. Flow Navigator → **Run Implementation**. Wait for completion. After implementation, click **Open Implemented Design** → **Report Timing Summary** and confirm WNS ≥ 0 ns and WHS ≥ 0 ns.
3. Flow Navigator → **Generate Bitstream**. Output: `ingest_top.bit`.
4. Connect the Nexys A7 via USB and power it on.
5. Open Hardware Manager → **Open Target** → **Auto Connect**.
6. Right-click the `xc7a100t` device → **Program Device**, point at `ingest_top.bit`, click **Program**. The DONE LED should illuminate.

[Figure placeholder]

`LH1H_timing_report.png`

Vivado implementation timing summary with positive WNS. Any negative slack must be resolved before programming the board.

Figure 3: Vivado implementation timing summary with positive WNS. Any negative slack must be resolved before programming the board.

5.5 Step 5 — End-to-end run

1. Confirm the wiring per the table on page 2: Nucleo Morpho `PC4` → PMOD JA pin 1, GND → PMOD JA pin 5.
2. Power on Target A and Target B (LH1-F). Confirm their USART2 logs show the ping-pong CSV lines.
3. Power on the Threat (LH1-G). Confirm its USART2 log shows `RX sender=01` and `RX sender=02` lines alternating once per second.
4. Observe the Nexys A7 seven-segment display. Right-most two digits should alternate `01` (Target A's DATA) and `02` (Target B's ACK) in step with each frame; left-most two digits should show a slowly-incrementing 16-bit frame counter (low byte). `LED[0]` (UART byte strobe) should flicker rapidly; `LED[1]` (new_frame pulse) should blink at the per-frame cadence; `LED[2]` (FIFO empty) should be on most of the time; `LED[3]` (FIFO full) should never light.
5. **Perturbation 1:** power off Target A. The display should lock to `02` for one frame and then go silent (Target B has nothing to ACK). Power Target A back on; alternating display should resume within 2 s.
6. **Perturbation 2:** hold the Threat user button (B1) for 3 s. The display should briefly fall silent (the targeted Target stops transmitting DATA, so the other has nothing to ACK), then resume alternating within ∼500 ms of release.

[Figure placeholder]

`LH1H_seg7_display.jpg`

Module 1 capstone shot: Nexys A7 seven-segment display showing a live Beacon ID captured from the full ***cooperative target*** → ***RF link*** → ***interceptor (Threat)*** → ***FPGA*** *chain.*

Figure 4: Module 1 capstone shot: Nexys A7 seven-segment display showing a live Beacon ID captured from the full **cooperative target** → **RF link** → **interceptor (Threat)** → **FPGA** chain.

6 Verification Checklist

- ☐ Vivado project `ingest_top` created with all five VHDL sources and one XDC; `ingest_top` marked as Top.
- ☐ Synthesis, Implementation, and Bitstream generation complete with zero errors.
- ☐ WNS ≥ 0 ns and WHS ≥ 0 ns in the implementation timing report.
- ☐ DONE LED illuminates after programming.
- ☐ `LED[0]` flickers (UART bytes arriving), `LED[1]` pulses at ∼1 Hz (new_frame), `LED[3]` (FIFO full) never lights.
- ☐ Seven-segment display alternates `01` and `02` matching the live Beacon IDs.
- ☐ Powering Target A off then on causes the display to silence then resume within 2 s.
- ☐ Holding the Threat's user button suspends the alternating display; releasing it restores normal operation within ∼500 ms.

Troubleshooting

- **No display change, `LED[0]` dark:** no UART bytes are reaching the FPGA. Re-check the PMOD wire (JA pin 1, not pin 2) and the GND wire. Verify with a scope on the FPGA side of the jumper that you see 3.3 V LVCMOS-level transitions when the Threat is transmitting.
- **`LED[0]` flickers but `LED[1]` stays dark:** bytes are arriving but no frame ever assembles. Likely the FIFO write or read enable is wrong, or the FSM never sees `0x7E`. Add an ILA probe on `rx_data` and look for `0x7E` bytes; if absent, your Threat is not actually emitting frames (re-validate LH1-G Step 7 with the Python parser).
- **Display garbled / wrong digits:** segment-anode mapping inverted. The Nexys A7 segments and anodes are *both active-low*; double-check your XDC and the `hex7` lookup table.
- **Display flickers unreadably:** multiplex rate too slow. The default REFRESH_DIV gives 4 kHz total / 1 kHz per digit; if you reduced it, raise it back.
- **Frame detector stuck (`LED[1]` never pulses) but `0x7E` bytes are visible on ILA:** length byte mismatch. The Threat must be sending `0x29` as byte 1 of every frame. If you see anything else, your Threat firmware has the wrong length constant.

Lab Handout 1-H Deliverable — and Module 1 Final Deliverable

1. All five VHDL sources: `uart_rx.vhd`, `byte_fifo.vhd`, `frame_detector.vhd`, `seg7_driver.vhd`, `ingest_top.vhd`.
2. Pruned `Nexys-A7-100T-Master.xdc` with only the active lines shown in Step 3 uncommented.
3. Vivado utilization and timing reports (the same `impl_summary.json` format from LH2-H is fine but not required at this stage; raw `.rpt` files are acceptable).
4. Short video (30–60 s) of the complete end-to-end system running: cooperative-target ping-

pong, Threat forwarding, Nexys A7 seven-segment display updating with Beacon IDs and the frame counter, and a button-hold demonstration showing the display going silent then resuming.

5. One-page Module 1 retrospective: what worked, what didn't, what you would change before Module 2.

Glossary

FIFO	First In, First Out	1
FPGA	Field-Programmable Gate Array	1, 2
RF	Radio Frequency	1
UART	Universal Asynchronous Receiver/Transmitter	1
VHDL	VHSIC Hardware Description Language	4